

Übungsaufgaben zur Wiederholung im Modul Programmierung

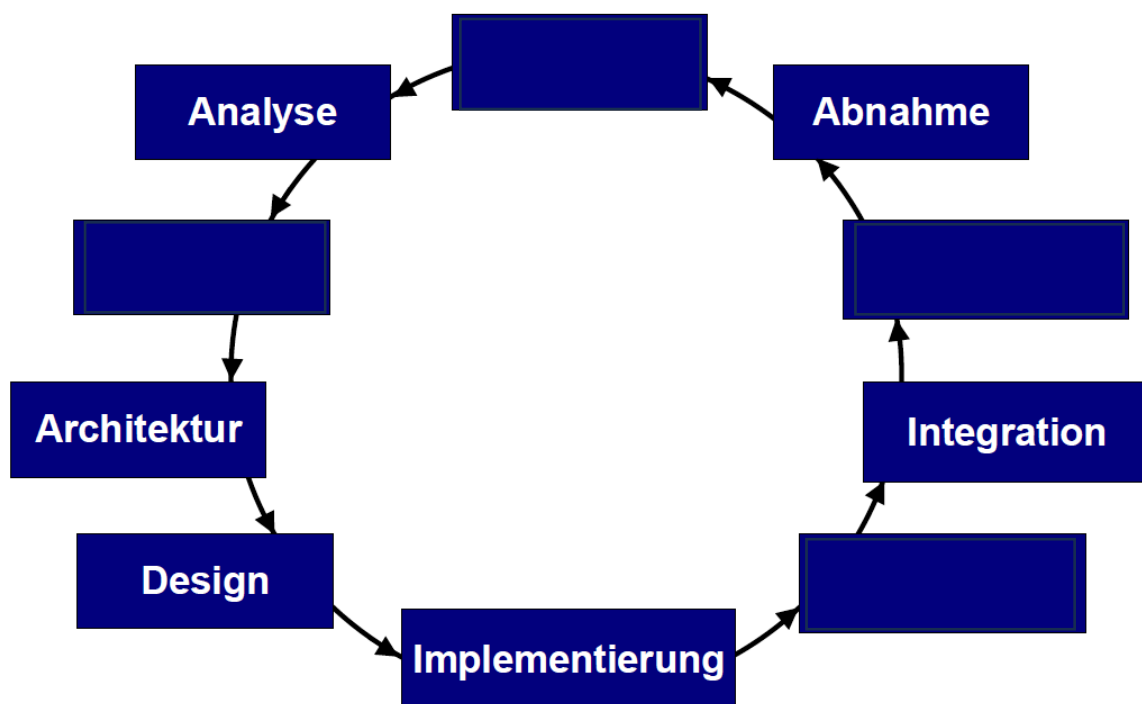
Aufgabe 1

In der Softwareentwicklung spielen Qualitätsmerkmale eine wesentliche Rolle, um sicherzustellen, dass die Software sowohl den Anforderungen des Benutzers als auch technischen Standards und Best Practices entspricht. Qualitätsmerkmale können aus der Perspektive des Endbenutzers oder des Entwicklers betrachtet werden.

- Listen Sie drei Qualitätsmerkmale auf, die für Benutzer besonders wichtig sind, und erläutern Sie kurz, warum diese aus Benutzersicht relevant sind.
- Listen Sie drei Qualitätsmerkmale auf, die aus der Sicht eines Softwareentwicklers besonders wichtig sind, und erläutern Sie kurz, warum diese aus Entwicklersicht relevant sind.
- Nennen Sie ein Qualitätsmerkmal, welches sowohl für den Benutzer als auch für den Softwareentwickler relevant ist.

Aufgabe 2

Ergänzen Sie die vier fehlenden Begrifflichkeiten in folgendem Bild:



Erläutern Sie stichpunktartig, welche Aufgaben in der jeweiligen Phase bearbeitet werden müssen und was das Ergebnis der Phase ist.

Aufgabe 3

Kreuzen Sie an: Was gehört ins Lastenheft und was ins Pflichtenheft?

	<i>Lastenheft</i>	<i>Pflichtenheft</i>
Beschreibung der allgemeinen Zielsetzung des Projekts.		
Detaillierte technische Spezifikationen für die Softwarelösung.		
Die Fristen für die Fertigstellung der verschiedenen Projektphasen.		
Eine Liste von Funktionen, die der Kunde von der Software erwartet.		
Informationen zur Softwarearchitektur und den zu verwendenden Technologien.		

Aufgabe 4

- a) Gegeben ist eine einfache Python-Funktion, die den Ticketpreis für eine Veranstaltung berechnet. Die Funktion nimmt das Alter des Käufers als Parameter entgegen und gibt den Preis des Tickets zurück.

```
def berechne_ticketpreis(alter):
    if alter < 0:
        return "Ungültiges Alter"
    elif alter <= 12:
        return 5.0 # Euro
    elif alter <= 65:
        return 10.0 # Euro
    else:
        return 7.5 # Euro
```

1. Die Funktion ist mit einem Black-Box-Test zu prüfen. Identifizieren Sie die Äquivalenzklassen für die Eingabe dieser Funktion. Erklären Sie kurz, warum Sie diese Klassen gewählt haben.
 2. Erstellen Sie Tests mithilfe von pytest, die die Funktion mit jeweils einem repräsentativen Wert aus jeder Äquivalenzklasse testet. Geben Sie das Ergebnis der Tests aus.
- b) Gegeben ist eine rekursive Python-Funktion, die die Fakultät einer gegebenen nicht-negativen Ganzzahl berechnet.

```
def fakultaet(n):
    if n < 0:
        return None
    if n == 0:
        return 1
    return n * fakultaet(n-1)
```

1. Die Funktion ist mit einem White-Box-Test zu prüfen. Welche Pfade sollten hierfür betrachtet werden? Bitte benennen und erläutern Sie diese kurz.
2. Erstellen Sie korrespondierenden Tests mithilfe von pytest.

Aufgabe 5

Schreiben Sie ein Python-Skript, das das `getopt`-Modul verwendet, um einen einfachen Taschenrechner zu entwickeln. Ihr Skript sollte die folgenden Optionen unterstützen:

- `-h` oder `--help`: Gibt eine Hilfe-Nachricht aus, die erklärt, wie das Skript verwendet wird.
- `-a` oder `--add`: Nimmt zwei Zahlen als Argumente und gibt ihre Summe aus.
- `-m` oder `--multiply`: Nimmt zwei Zahlen als Argumente und gibt ihr Produkt aus.
- Die Zahlen sollen als `-x / --zahlx` bzw. `-y / --zahly` übergeben werden

Beispiel:

```
python myscript.py --add -x 5 -y 10          # Sollte "15" ausgeben
python myscript.py -m --zahlx 3 --zahly 7    # Sollte "21" ausgeben
python myscript.py -h                        # Sollte die Hilfe-Nachricht ausgeben
```

Anforderungen:

1. Ihr Skript sollte sowohl die kurzen als auch die langen Formen der Optionen unterstützen (z.B. `-h` oder `--help`).
2. Das Skript sollte angemessene Fehlermeldungen ausgeben, wenn ungültige Optionen oder Argumente angegeben werden.

Aufgabe 6

Schreiben Sie ein Python-Programm, das die Textdatei namens `textdata.txt` einliest. Das Programm soll die folgenden Aufgaben durchführen:

1. Zählen Sie die Anzahl der Wörter in der Datei und geben Sie diese aus.
2. Erstellen Sie ein Wörterbuch, das die Häufigkeit jedes Wortes in der Datei enthält. Ein Wort soll hier als eine Folge von Buchstaben betrachtet werden, die durch Leerzeichen oder Satzzeichen getrennt sind.
3. Finden Sie das am häufigsten vorkommende Wort in der Datei und geben Sie das Wort zusammen mit seiner Häufigkeit aus.
4. Schreiben Sie das Ergebnis der Berechnungen in eine Textdatei mit dem Namen „`ergebnisse.txt`“

Inhalt von `textdata.txt`:

```
Das ist ein Test. Dieser Test ist nur ein Beispiel.
```

Beispielausgabe:

```
Anzahl der Wörter: 10
```

Häufigkeit der Wörter: {'Das': 1, 'ist': 2, 'ein': 2, 'Test': 2, 'Dieser': 1, 'nur': 1, 'Beispiel': 1}
Am häufigsten vorkommendes Wort: 'ist' (2 Mal)

Aufgabe 7

Erstellen Sie eine einfache Aufgabenverwaltung. Eine Aufgabe besteht aus einer kurzen textuellen Beschreibung und einer Priorität:

```
class Aufgabe:
    def __init__(self, beschreibung, prioritaet):
        self.beschreibung = beschreibung
        self.prioritaet = prioritaet
```

Bitte implementieren Sie die Datenstruktur der Aufgabenverwaltung intern als doppelt verkettete Liste. Die Applikation soll folgende Funktionen beinhalten:

- Aufgabe mit bestimmter Beschreibung und Priorität hinzufügen
- Aufgabe nach einer bestimmten Aufgabe in die Liste einfügen
- Liste mit allen Aufgaben zurückgeben
- Sortierung der Liste

Nutzen Sie für die Implementierung der Sortierung der Liste den „Insertion Sort“-Ansatz.

Aufgabe 8

Gegeben ist der folgende unvollständige Code eines Binären Suchbaums.

prog_ss24/src/aufgabe05.py

```
class Node:
    def __init__(self, key):
        self.left = None
        self.right = None
        self.val = key

class BinarySearchTree:
    def __init__(self):
        self.root = None

    def insert(self, key):
        """
        Fügt ein neues Element mit dem gegebenen Schlüssel ein.
        """
        if self.root is None:
            self.root = Node(key)
        else:
            self._insert_rekursiv(self.root, key)
```

```
def _insert_rekursiv(self, node, key):
    pass

def print_tree(self):
    """
    Gibt die Keys des Baums in aufsteigender Reihenfolge aus
    """
    pass

# Verwendung
bst = BinarySearchTree()
elements = [20, 10, 30, 5, 15, 25, 35]
for elem in elements:
    bst.insert(elem)

bst.print_tree()
```

- a) Bitte implementieren Sie die Funktion `_insert_rekursiv` zum rekursiven Einfügen von neuen Elementen im Binären Suchbaum
- b) Bitte implementieren Sie die Funktion `print_tree`

Aufgabe 9

Sie sind in einem Unternehmen tätig und müssen die Mitarbeiter nach Jahresgehalt sortieren. Ihre Aufgabe besteht darin, eine Funktion **mergesort** zu erstellen, die eine aufsteigende Sortierung nach Gehalt vornimmt.

Beispiel:

```
mitarbeitergehaelter = [
    {'name': 'Klaus', 'gehalt': 270000},
    {'name': 'Peter', 'gehalt': 150000},
    {'name': 'Markus', 'gehalt': 350000},
    {'name': 'Patrick', 'gehalt': 500000},
    {'name': 'Johannes', 'gehalt': 100000}
]
print(mergesort(mitarbeitergehaelter))
```

Erwartetes Ergebnis:

```
[
    {'name': 'Johannes', 'gehalt': 100000},
    {'name': 'Peter', 'gehalt': 150000},
    {'name': 'Klaus', 'gehalt': 270000},
    {'name': 'Markus', 'gehalt': 350000},
    {'name': 'Patrick', 'gehalt': 500000}
]
```